Proactive Aging Mitigation in CGRAs through Utilization-Aware Allocation

Marcelo Brandalero*, Bernardo Neuhaus Lignati†, Antonio Carlos Schneider Beck†,
Muhammad Shafique‡, Michael Hübner*,
* *Chair of Computer Engineering, Brandenburg University of Technology (B-TU).* Cottbus, Germany.
† *Institute of Informatics, Universidade Federal do Rio Grande do Sul (UFRGS).* Porto Alegre, Brazil.
‡ *Institute of Computer Engineering, Vienna University of Technology (TU Wien).* Vienna, Austria.

***Abstract*—Resource balancing has been effectively used to mitigate the long-term aging effects of Negative Bias Temperature Instability (NBTI) in multi-core and Graphics Processing Unit (GPU) architectures. In this work, we investigate this strategy in Coarse-Grained Reconfigurable Arrays (CGRAs) with a novel application-to-CGRA allocation approach. By introducing important extensions to the reconfiguration logic and the datapath, we enable the dynamic movement of configurations throughout the fabric and allow overutilized Functional Units (FUs) to recover from stress-induced NBTI aging. Implementing the approach in a resource-constrained state-of-the-art CGRA reveals $2.2\times$ lifetime improvement with negligible performance overheads and less than 10% increase in area.**

***Index Terms*—reconfigurable systems, Coarse-Grained Reconfigurable Arrays (CGRAs), aging, utilization-aware, mapping, allocation.**

I. Introduction

Hardware aging has emerged as a critical reliability threat that can lead to degraded performance and early-stage system failure [1]. One of the main phenomena leading to increased Threshold Voltage (V_t), circuit delays, and device wear-out is Negative-Bias Temperature Instability (NBTI), caused by the massive stress induced on the PMOS transistors [2], [3]. Partial recovery from this aging threat is observed when the circuit is power-gated or remains idle for a given time. Utilization-aware resource balancing can thus be leveraged to alleviate the long-term aging effects of NBTI. Recent approaches to aging mitigation exploit the architectural regularity in Chip Multi-Processors (CMPs) and Graphics Processing Units (GPUs) with scheduling algorithms that balance the utilization of the processing resources (i.e., cores) and reduce the accumulated stress, thereby slowing down the wear-out [3]–[10]. However, within the scope of small low-power CMPs or even single-core embedded systems, the absence of a regular structure hinders the application of similar strategies.

Reconfigurable architectures, in particular, Coarse-Grained Reconfigurable Array (CGRA) systems, offer a promising solution in that direction. Besides being dynamically customizable to implement different application datapaths that improve performance and energy efficiency, CGRAs also offers a regular structure that can be leveraged by utilization-balancing strategies for aging mitigation. While previous works have addressed energy-efficient design and mapping strategies for CGRAs [11]–[17], aging mitigation in such architectures has received significantly less attention, with the proposed strategies being either oblivious to the utilization [18] or requiring changes in the compiled code [19]. To address these limitations in CGRAs, in this paper, we propose an automatic approach for aging mitigation through utilization-aware resource balancing.

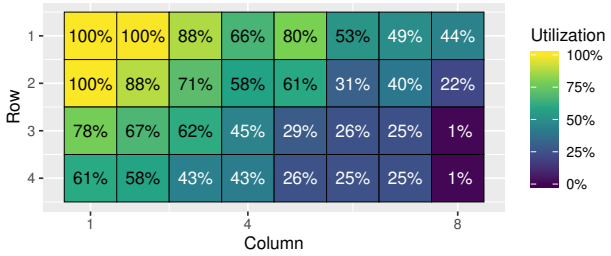


Fig. 1. Utilization rate of the FU in a 1D (only left-to-right data propagation) 4x8 CGRA design when employing traditional mapping methods and executing a set of embedded applications.

Motivational Analysis: In CGRAs, when employing traditional energy-efficient application mapping strategies [12], [13], [17], the allocation of the operations is typically biased towards one of the corners of the array. The reason for that is, the allocation algorithms greedily select the first available FU where an operation may execute in order to minimize the execution time, for instance, through reduced communication time. Fig. 1 from our experiments illustrates this phenomena by showing the average utilization of the FUs in a rectangular CGRA fabric (such as in [15], [16], [20]) when executing a set of embedded benchmark applications. As can be seen, the top-left-most FU is used by 100% of the CGRA configurations, while the bottom-right-most one is used by only 1% of the total configurations. As a consequence of their high utilization, FUs around the top-left corner undergo more stress phases over time and can age up to $10\times$ faster, leading to early-stage FU failures that limit the Instruction-Level Parallelism (ILP) exploitation and CGRA performance.

Proposed Approach: Ideally, the utilization should be uniformly distributed across the CGRA's FUs to ensure a uniform aging rate and an extended system lifetime. The FUs with low utilization thus represent a *utilization budget* that can be leveraged to slow down aging in the most stressed components. Towards this goal, we propose a novel configuration allocation procedure that supports the automatic run-time movement of CGRA configurations through the reconfigurable fabric, improving over traditional aging-unaware allocation strategies. The approach is implemented by deploying important,

978-1-7281-1085-1/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on July 26,2026 at 09:34:06 UTC from IEEE Xplore. Restrictions apply.

yet low-cost extensions to the CGRA's reconfiguration logic and datapath that allow configuration movement without any significant performance overhead, effectively distributing the FU's utilization more uniformly across the fabric.

In summary, this work makes the following contributions:

- We propose a novel utilization-aware configurations allocation strategy for CGRAs that automatically slows down the NBTI aging effects by balancing the utilization of the FUs (Section III.A).
- We introduce the required architectural extensions to support the proposed allocation approach in a state-of-art CGRA [20] (Section III.B);
- We show how our strategy can increase the CGRA's lifetime by 2.4× even under resource-constrained scenarios while introducing negligible performance and less than 10% area overhead (Section IV).

II. Background and Related Work

A. Hardware Aging

NBTI is one of the fundamental aging phenomena affecting PMOS transistors. Setting V_{gs} to $-V_{dd}$ (i.e. switching the transistor on) leads to an increase in the V_t which is commonly referred to as *short-term aging*. When the stress is released by setting V_{gs} to 0, the increase in V_t is only *partially* recovered, thereby leading to a continuous delay degradation over a long period of time (i.e., *long-term aging*) [2], [4], [21]. Recent evaluations suggest nearly 10% increase in circuit delay after 3 years [22] and 20% increase after 10 years [23], or complete wear-out in less than 3 years even for very low stress rates [5].

The default strategy to prevent premature system failure is to ship designs with timing guardbands, setting a nominal frequency lower than the maximum one and accounting for the wear-out effects over several years [24], [25]. Architectural-level aging-mitigation strategies usually employ utilization balancing of the processing resources. For that, however, a regular structure is required, limiting the application of the approach to multi-core [4]–[6] and GPU [3], [7], [8] architectures.

In this work, we use a predictive model for NBTI aging based on a prominent related work [26] that gives the long-term NBTI-induced V_t as a function of the Operating Voltage (V_{dd}), Temperature (T), time (t) and duty cycle (d, which is equivalent to the utilization rate u of an FU) – see Eq. 1. The increase in delay can then be approximated to first order as the relative increase in V_t.

$$\Delta V_t = 0.005 \times e^{-1500/T} \times V_{dd}^4 \times t^{1/6} \times u^{1/6} \qquad (1)$$

B. Coarse-Grained Reconfigurable Arrays

Reconfigurable architectures, in particular, CGRAs, represent an attractive solution for energy-efficient execution even for single-threaded applications since they enable hardware customization at run time to match different computational requirements [27], [28]. Application code can be mapped to CGRAs either *statically* [11], [13] (at compile time) or *dynamically* [14]–[17] (at run time). Dynamic mapping approaches present several advantages. First, they enable the automatic acceleration of binaries after deployment, without the need to recompile. Second, they can leverage dynamic information (such as how often each application region is executed) for optimizing the configurations towards the energy-efficiency target.

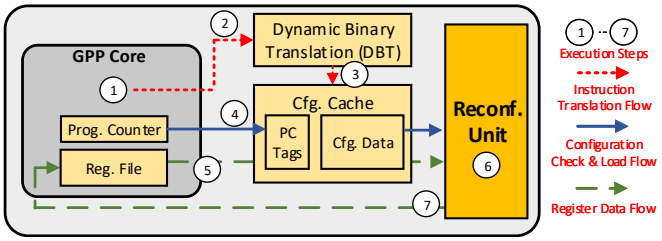


Fig. 2. Overview of the TransRec system [20], which serves as a baseline for the implementation of the utilization-aware allocation proposed in this work.

An example of such a system supporting dynamic mapping is the TransRec system [20], which we use as a baseline for the proposed aging-aware load balancing strategy (described next in Section III). TransRec consists of a General-Purpose Processor (GPP) core, a tightly-coupled CGRA-based reconfigurable unit, and a hardware-implemented Dynamic Binary Translation (DBT) module that automatically transforms binary code at run time into configurations for CGRA execution. An overview of this system and its execution process is presented in Fig. 2 and detailed next. An application begins its execution on the GPP core (Step 1). As instructions finish their execution, they are sent to the DBT module (Step 2), which interprets their semantics, finds the dependencies among them, and allocates them into a CGRA configuration. A CGRA configuration is composed of a sequence of instructions that are then saved in a dedicated configuration cache and indexed by the PC of the first instruction of that sequence, for posterior acceleration (Step 3). Therefore, while the GPP executes the application, the DBT also continually checks the configuration cache for a matching configuration for the next instruction sequence using the Program Counter (PC) (Step 4). When entry one is found, it is offloaded from the configuration cache along with the input register values coming from the GPP (Step 5) and executed in the CGRA (Step 6) After the execution is completed, the output registers' results are written back and committed *in program order* to the GPP (Step 7). This execution model enables on-the-fly acceleration without changing the application binary.

We present the detailed architecture of TransRec's reconfigurable unit in Section III, along with the required extensions to support the proposed aging-mitigation approach.

C. Research Opportunities

There is only a limited body of work on aging mitigation in CGRAs. Previous work has modified a static mapping strategy for aging-aware allocation [19], or used aggressive voltage underscaling to achieve low energy consumption while also reducing aging [18]. This static approach, however, is limited to new applications with availability of source code, requires redeployment, and is unaware of dynamic input-dependent

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on July 26,2026 at 09:34:06 UTC from IEEE Xplore. Restrictions apply.

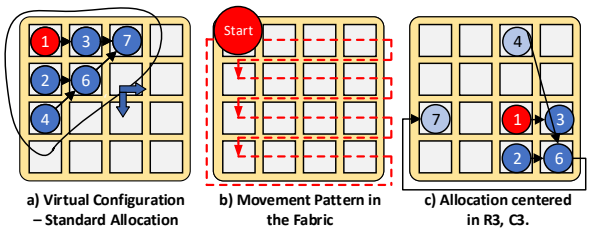


Fig. 3. Overview of the two-step *configuration generation* and *configuration allocation* approach.

information that affects the execution. Another work uses voltage aggressive underscaling to achieve low energy consumption and also reduce aging, given the dependence of NBTI on the supply voltage [18]. The approach proposed in this work is complementary to [18] since NBTI is addressed by balancing the utilization of the processing resources and therefore increasing the stress-to-recovery ratio, which affects aging as per Eq. 1.

III. Proposed Utilization-Aware Allocation Strategy

As described in Section II, current application-to-CGRA mapping approaches are limited in their ability to address NBTI aging. In an ideal scenario, an aging-aware strategy should not allocate the application's instructions onto the lowest-health FUs in the CGRA fabric. However, detecting the optimal allocation at run time may turn out to be prohibitively expensive; while supporting random allocations (and therefore achieve a uniform distribution over time) in the CGRA fabric with a complex interconnection network may severely impact performance.

We propose a lightweight yet effective alternative: a utilization-aware configuration allocation strategy that enables automatic aging mitigation. The strategy is implemented directly in the hardware, and therefore requires no changes to the precompiled CGRA binaries or dynamic mapping strategies. An overview is provided in Fig. 3. Given a previously generated CGRA configuration (using static or dynamic methods), which we refer to as a *virtual configuration* (See Fig. 3a), the allocation approach consists of moving this configuration through the fabric horizontally and vertically whenever a new execution takes place. To do so, we move the position of the configuration *pivot* (red circle in Fig. 3) for each new execution following the pattern depicted in Fig. 3, which covers all of the reconfigurable fabric. By moving the *pivot*, the entire configuration moves with it, and a more uniform utilization of the entire fabric should be achieved. To cover the entire CGRA, *wrap-around* as shown in Fig. 3c is also supported.

We proceed with a description of the TransRec CGRA, which serves as a use-case on top of which we implement the proposed approach. After that, we show the required architectural extensions to support the proposed approach.

A. Baseline Implementation

The TransRec CGRA is a matrix of FUs composed entirely of combinational logic and divided into rows and columns – see Fig. 4. Data propagates from left to right, so that each FU occupies a row and a sequence of columns (according to its latency). Since the design is highly regular, it is easily configurable for exploiting distinct ranges of ILP. For the technology under consideration in this work, Arithmetic-Logic Units (ALUs) have the latency of half a processor cycle due to their simplicity and correspond to a single column. Loads and stores are constrained by the data cache, with one read and one write, and take two processor cycles to complete (or, equivalently, four columns). As shown in the figure, while a single load operation or store is executed, two chains of four data-dependent ALU operations may be executed concurrently.

FUs communicate via context lines, initially fed by values from the input context. Before each FU, a crossbar selects the context line that feeds the inputs. After each FU, another crossbar selects, for each context line, which values from the FUs of the current column are propagated to the next ones.

B. Microarchitectural Extensions

To implement the instruction rotation mechanism with no performance overheads, we need to modify the interconnection network to allow:

- the allocation of a previously-generated configuration into the CGRA fabric to start at an arbitrary column and row;
- the allocation to "*wrap-around*", so that operations in the last row and column can propagate their results to the first row and column (similarly to how circular buffers are designed).

Moreover, the implementation of the approach requires changes both in the reconfiguration logic and in the datapath. Two operations must be supported: *vertical movement* and *horizontal movement*.

For the *vertical movement* operation, we need to only change the CGRA's reconfiguration logic. This is the structure shown in Fig. 5a, which is responsible for receiving the configuration bits from the configuration cache and forwarding them to the appropriate columns and rows. It works as follows. Input to the reconfiguration logic are n *configuration lines*, each of which contains the reconfiguration bits for an entire column. Columns continuously monitor the *configuration lines* such that column i is connected to line $i \ mod \ n$. For $n = 4$, for instance (as in Fig. 5), columns $0, 4, 8 \ldots$ are connected to bus 0, columns $1, 5, 9, \ldots$ to bus 1 and so on. A *control unit* sends *write* signals so that the context registers for columns $i, \ldots, i{+}n$ are reconfigured in cycle i. In the example, columns $1 \ldots 4$ would all be reconfigured in the first cycle.

In the implementation supporting the configuration movement approach, the reconfiguration logic is extended with multiplexers to allow each column i to fetch the values in any of the n configuration lines. Fig. 5b shows this new multiplexer (in purple), which allows the column to receive the configuration bits for any of the configuration lines, not only the first one (shown in red). This microarchitectural change enables the horizontal movement of the configurations through the fabric.

Lastly, Fig. 5c shows (in purple) the required extensions

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on July 26,2026 at 09:34:06 UTC from IEEE Xplore. Restrictions apply.

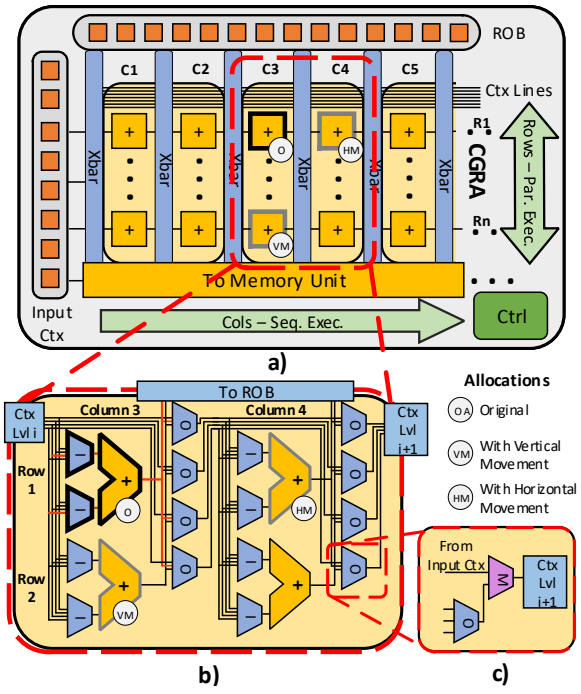


Fig. 4. Detailed view of (a) the CGRA-based Reconfigurable Unit, (b) two columns with reconfigurable datapath and the option to move the original FU allocation (O) vertically (VM) and horizontally (HM), and (c) the required addition to the datapath to support horizontal movement (shown with R).

necessary to support the vertical movement of the configuration. Considering that the input configuration bits are responsible for an entire column, which includes the input multiplexers, the FUs and the output multiplexers, we extend the microarchitecture with barrel shifters for each of these structures (shown in purple). Although barrel shifters are known to introduce significant latency overheads, the number of rows in such a CGRA is usually small (≤ 8) due to the limited application ILP, so such overhead is only marginal, as our results will show.

The proposed approach, in particular, the horizontal movement operation, also requires the introduction of one new set of multiplexers for each column and the context line in the datapath. An additional 2:1 multiplexer (shown in purple in Fig. 4) selects between the previous line and the initial input context. This multiplexer is deployed to enable the wrap-around operation, i.e., so that a configuration may start the execution from an arbitrary column. The multiplexer from the last column then feeds back into the first one to enable the wrap-around of the operations (i.e., operations in the last column can communicate to the first column).

IV. Evaluation Methodology

We evaluate the proposed utilzation-aware allocation for its lifetime-extension capabilities and its performance and area overheads, comparing these results to the utilization-unaware implementation. We target the RISC-V Instruction-Set Architecture (ISA) due to the availability of open Hardware Description Language (HDL) processor designs that allow for high-accuracy area and power estimations.

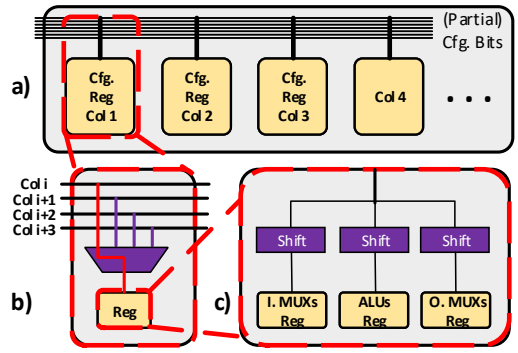


Fig. 5. Detailed view of the reconfiguration logic, with (a) the configuration lines feeding all columns, (b) a column listening to a particular configuration line, and (c) the configuration bits feeding to the input multiplexers, FUs and output multiplexers.

A. Tools and Design Flow

For performance evaluation and fast design-space exploration, we extended the gem5 cycle-accurate simulator [29] with the TransRec implementation Binary Translation (BT) and CGRA designs based on [20], using the *TimingSimple* CPU to model a single-issue core. 10 benchmarks[1] from mibench [30], typically found in the embedded domain, compiled for the RISC-V ISA with -O3 and running the "small input set" were used in the evaluation.

For area and energy evaluation, an HDL prototype of the system built on top of the Rocket core [31] was developed and synthesized with Cadence RTL Compiler and NanGate's 15nm standard cell library [32]. Results for the caches were estimated using FinCACTI [33].

Finally, to evaluate the aging, the model described earlier in Section II (Eq. 1) was used to estimate the impact of utilization on the NBTI-induced increase in V_t. The V_t degradation causes a linear increase in delay, according to the model of [26]. We consider that the product's end-of-life is determined by the aging rate of the most stress-induced component in the design (i.e., the FU with the highest utilization). A worst-case delay degradation of 10% over 3 years was considered as estimated in the literature [23], [34].

B. Design Space Exploration

An initial exploration of the unmodified TransRec architecture design space was carried out to select interesting design points to evaluate the aging effects. This exploration covered different sizes of the CGRA fabric (length as number of columns L, representing sequential execution, varied from 8 to 32, and width as number of rows W, representing parallel execution, varied from 2 to 8) and its impact onto the execution time, the energy consumption and the average utilization of the FUs (all compared to a stand-alone Rocket GPP core). Fig. 6 shows the result of this exploration. The red square

[1]The following subset of *mibench* was selected due compatibility with the simulation toolflow: *bitcount*, *CRC32*, *dijkstra*, *qsort*, *rijndael-e*, *sha*, *stringsearch*, *susan* (corners, edges and smoothing).

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on July 26,2026 at 09:34:06 UTC from IEEE Xplore. Restrictions apply.

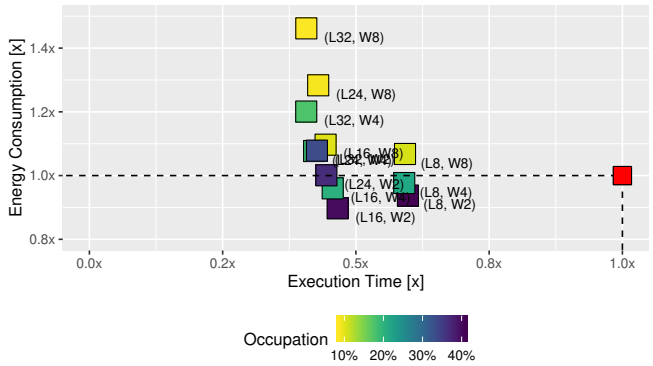


Fig. 6. System-wide performance, energy consumption and average occupancy for different sizes of CGRA fabric.

with $1\times$ represents the performance and energy consumption of the GPP alone, and the others the TransRec system. From this analysis we select the following designs of interest:

- $(L16, W2)$, with $2.14\times$ speedup, 10% reduction in energy consumption and an average utilization of 39.7%. We name this scenario BE (*best energy consumption*).
- $(L32, W4)$, with $2.45\times$ speedup, 20% increase in energy consumption and an average utilization of 17.8%. We name this scenario BP (*best performance*).
- $(L32, W8)$, with $2.45\times$ speedup (same as above), 46% increase in energy consumption and an average utilization of 8.9%. We name this scenario BU (*best (lowest) utilization*).

V. Results and Discussion

A. Utilization and Lifetime Extension

Fig. 7 shows the utilization of the CGRA fabric in the *BE* scenario, for both the baseline allocation strategy (on the top) as well as the utilization-aware allocation (on the bottom). As initially noted in Section I, the lower columns and rows present a higher utilization than the upper ones when employing traditional allocation strategies. With the proposed utilization-aware allocation, the utilization is more evenly distributed across the fabric, avoiding high stresses in a single component. The maximum utilization drops from 94.5% in the baseline to 41.2% in the proposed approach.

Due to space limitations, rather than presenting the same figure for the *BP* and *BU* scenarios (which represent larger CGRA fabrics), we show in the upper part of Fig. 8 the *probability density function* for the utilization of a FU in the three scenarios, for both the baseline as well as the proposed allocation approach. These plots show that larger fabrics such as in the *BU* scenario have a significant amount of FU with small utilization when using the baseline allocation, increasing the potential of the proposed approach.

Given these distributions and the highest utilization for each scenario (which ultimately determines the first component to fail), we use Eq. 1 to estimate the improvements in the aging rate of the design. The lower part of Fig. 8 shows the delay increase over time due to NBTI aging for the baseline and proposed approaches. Larger designs present the

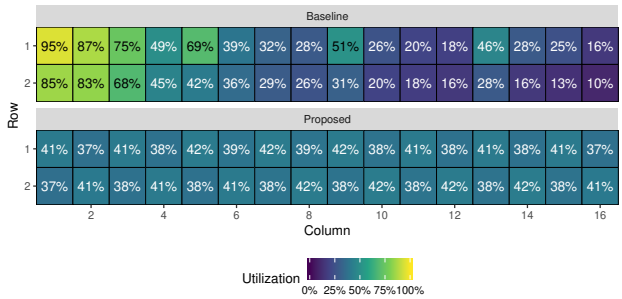


Fig. 7. Average utilization of each FU in the traditional and proposed approaches (left side) along with probability density plot for the utilization rate. Results for the BE scenario (16x2 design).

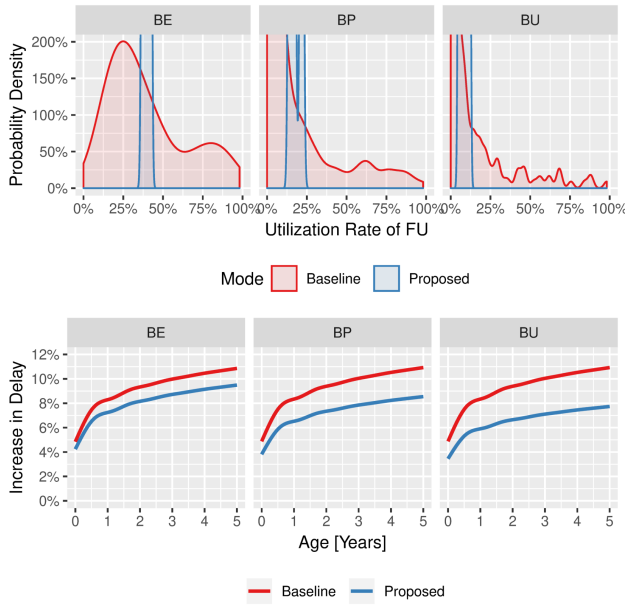


Fig. 8. NBTI-induced increase in delay over the years for different CGRA sizes and considering the utilization-unware allocation and the proposed one.

highest difference, as they offer a higher *utilization budget* for balancing. For the smallest design (*BE*), the aging rate is slowed down by $2.29\times$, and the system presents a performance degradation of 10% only in 7 years rather than in 3. Larger designs lead to even better improvements in the product's lifetime as they offer even more resources for utilization balancing. Table I summarizes these findings.

B. Area Overhead

As described in Section III, the proposed approach requires extending the reconfiguration logic and the datapath. Table II presents the area results for the implementation of the *BE* scenario with and without the architectural extensions. The area overhead was found to be below 10%. Moreover,

TABLE I
Utilization and Lifetime Improvements for Each Scenario.

Scenario	Avg. Util	Baseline Worst Util.	Proposed Worst Util.	Lifetime Improv.
BE	39.7%	94.5%	41.1%	2.29×
BP	17.10%	98.1%	22.4%	4.37×
BU	8.5%	98.1%	12.3%	7.97×

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on July 26,2026 at 09:34:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
CGRA AREA OVERHEAD (BE SCENARIO).

	Baseline	Modified
Area[μ^2]	$28,995$	$30,199$ $(+4.15\%)$
# Cells	$79,540$	$83,083$ $(+4.45\%)$

considering only a single column in the design, both the baseline and the proposed version were able to reach the same minimum latency of 120ps (for one column, as shown in the left side of Fig. 4b). These results suggest that the introduced modifications do not affect the maximum frequency of the design, which is likely to run with a frequency below the maximum one due to power consumption constraints.

VI. CONCLUSIONS AND FUTURE WORK

In this work, we have proposed a utilization-aware allocation strategy for aging mitigation in CGRAs. The strategy is based on rotating the *virtual configurations*, which were originally targeted for performance/energy efficiency, throughout the fabric to balance the stress-to-recovery rates of the individual FUs, and thereby reduce the aging rate of the whole design. To enable this, the configuration bits are shifted at configuration load time by using the structures shown in Fig. 5. By doing so, our approach increases the lifetime of the design by $2.29\times$–$7.97\times$ for different design sizes. As a future work, we will implement the improved rotation techniques and use run-time aging information to adapt the allocation strategy dynamically. We will also evaluate homogeneous and heterogeneous multi-core scenarios.

ACKNOWLEDGEMENTS

This work is supported in part by the German Research Foundation (DFG) as part of the priority program "Dependable Embedded Systems" (SPP1500 - http://spp1500.itec.kit.edu), and in part by the Postoc Network Brandenburg (https://www.postdoc-network-brandenburg.de/en/).

REFERENCES

[1] J. Henkel, L. Bauer, J. Becker *et al.*, "Design and Architectures for Dependable Embedded Systems," in *CODES+ISSS*, 2011, pp. 69–78.
[2] K. Bernstein, D. J. Frank, A. E. Gattiker *et al.*, "High-performance CMOS variability in the 65-nm regime and beyond," *IBM Journal of Research and Development*, vol. 50, no. 4.5, pp. 433–449, 2006.
[3] A. Rahimi, L. Benini, and R. K. Gupta, "Hierarchically Focused Guardbanding: An adaptive approach to mitigate PVT variations and aging," in *DATE*, 2013, pp. 1695–1700.
[4] D. Gnad, M. Shafique, F. Kriebel *et al.*, "Hayat: Harnessing Dark Silicon and variability for aging deceleration and balancing," in *DAC*, 2015, pp. 1–6.
[5] F. Paterna, A. Acquaviva, and L. Benini, "Aging-Aware Energy-Efficient Workload Allocation for Mobile Multimedia Platforms," *IEEE Transactions on Parallel and Distributed Systems*, vol. 24, no. 8, pp. 1489–1499, 2013.
[6] T. R. Mück, Z. Ghaderi, N. D. Dutt *et al.*, "Exploiting Heterogeneity for Aging-Aware Load Balancing in Mobile Platforms," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 3, no. 1, pp. 25–35, 2017.
[7] X. Chen, Y. Wang, Y. Liang *et al.*, "Run-Time Technique for Simultaneous Aging and Power Optimization in GPGPUs," in *DAC*, 2014, pp. 168:1–168:6.
[8] H. Lee, M. Shafique, and M. A. Al Faruque, "Low-overhead Aging-aware Resource Management on Embedded GPUs," in *DAC'14*, 2017, pp. 67:1–67:6.
[9] S. Feng, S. Gupta, A. Ansari *et al.*, "Maestro: Orchestrating lifetime reliability in chip multiprocessors," in *International Conference on High-Performance Embedded Architectures and Compilers*. Springer, 2010, pp. 186–200.
[10] C. Bolchini, L. Cassano, and A. Miele, "Lifetime-aware load distribution policies in multi-core systems: an in-depth analysis," in *DATE'16*, 2016, pp. 804–809.
[11] M. Hamzeh, A. Shrivastava, and S. Vrudhula, "EPIMap: Using Epimorphism to map applications on CGRAs," in *DAC*, 2012, pp. 1280–1287.
[12] L. Chen and T. Mitra, "Graph Minor Approach for Application Mapping on CGRAs," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 7, no. 3, pp. 21:1–21:25, 2014.
[13] S. Dave, M. Balasubramanian, and A. Shrivastava, "RAMP: Resource-Aware Mapping for CGRAs," in *DAC*, 2018, pp. 1–6.
[14] N. Clark, M. Kudlur, S. Mahlke *et al.*, "Application-Specific Processing on a General-Purpose Core via Transparent Instruction Set Customization," in *MICRO*. IEEE, 2004, pp. 30–40, iSSN: 1072-4451.
[15] A. C. S. Beck, M. B. Rutzig, and L. Carro, "A transparent and adaptive reconfigurable system," *Microprocessors and Microsystems*, vol. 38, no. 5, pp. 509–524, 2014.
[16] F. Liu, H. Ahn, S. R. Beard *et al.*, "DynaSpAM : Dynamic Spatial Architecture Mapping using Out of Order Instruction Schedules," in *ISCA*, 2015, pp. 541–553.
[17] M. A. Watkins, T. Nowatzki, and A. Carno, "Software transparent dynamic binary translation for coarse-grain reconfigurable architectures," in *HPCA*, 2016, pp. 138–150.
[18] H. Afzali-Kusha, O. Akbari, M. Kamal *et al.*, "Energy Consumption and Lifetime Improvement of Coarse-Grained Reconfigurable Architectures Targeting Low-Power Error-Tolerant Applications," in *GLSVLSI*, 2018, pp. 431–434.
[19] Jiangyuan Gu, S. Yin, and S. Wei, "Stress-aware loops mapping on CGRAs with considering NBTI aging effect," in *DAC*, 2017, pp. 1–6.
[20] M. Brandalero, M. Shafique, L. Carro *et al.*, "TransRec: Improving Adaptability in Single-ISA Heterogeneous Systems with Transparent and Reconfigurable Acceleration," in *DATE*, 2019, pp. 582–585.
[21] M. A. Alam and S. Mahapatra, "A comprehensive model of PMOS NBTI degradation," *Microelectronics Reliability*, vol. 45, no. 1, pp. 71–81, 2005.
[22] F. Oboril and M. B. Tahoori, "ExtraTime: Modeling and analysis of wearout due to transistor aging at microarchitecture-level," in *DSN*, 2012, pp. 1–12.
[23] A. Tiwari and J. Torrellas, "Facelift: Hiding and slowing down aging in multicores," in *MICRO*, 2008, pp. 129–140.
[24] A. Rahimi, L. Benini, and R. K. Gupta, "Aging-aware compiler-directed VLIW assignment for GPGPU architectures," in *DAC*, 2013, pp. 1–6.
[25] C. R. Lefurgy, A. J. Drake, M. S. Floyd *et al.*, "Active Guardband Management in Power7+ to Save Energy and Maintain Reliability," *IEEE Micro*, vol. 33, no. 4, pp. 35–45, 2013.
[26] J. Henkel, T. Ebi, H. Amrouch *et al.*, "Thermal management for dependable on-chip systems," in *ASP-DAC'13*, Jan 2013, pp. 113–118.
[27] K. Compton and S. Hauck, "Reconfigurable computing: a survey of systems and software," *ACM Computing Surveys*, vol. 34, no. 2, pp. 171–210, 2002, publisher: ACM.
[28] M. Wijtvliet, L. Waeijen, and H. Corporaal, "Coarse Grained Reconfigurable Architectures in the Past 25 Years: Overview and Classification," in *SAMOS*, 2016, pp. 235–244.
[29] N. Binkert, S. Sardashti, R. Sen *et al.*, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, p. 1, 2011.
[30] M. Guthaus, J. Ringenberg, D. Ernst *et al.*, "MiBench: A free, commercially representative embedded benchmark suite," in *Fourth Annual IEEE International Workshop on Workload Characterization.*, 2001, pp. 3–14.
[31] K. Asanović, R. Avizienis, J. Bachrach *et al.*, "The Rocket Chip Generator," EECS Department, University of California, Berkeley, Tech. Rep., 2016.
[32] M. Martins, J. M. Matos, R. P. Ribas *et al.*, "Open Cell Library in 15nm FreePDK Technology," in *International Symposium on Physical Design (ISPD)*, 2015, pp. 171–178.
[33] A. Shafaei, Y. Wang, X. Lin *et al.*, "FinCACTI: Architectural Analysis and Modeling of Caches with Deeply-Scaled FinFET Devices," in *ISVLSI*, 2014, pp. 290–295.
[34] S. Rehman, F. Kriebel, Duo Sun *et al.*, "dTune: Leveraging reliable code generation for adaptive dependability tuning under process variation and aging-induced effects," in *DAC*, 2014, pp. 1–6.

Authorized licensed use limited to: Sungkyunkwan University. Downloaded on July 26,2026 at 09:34:06 UTC from IEEE Xplore. Restrictions apply.